

Code:

```

import java.util.*;

class Node {
    int nodeId;
    int parentNode;
    boolean hasToken;
    Queue<Integer> requestQueue;

    public Node(int nodeId, int parentNode, boolean hasToken) {
        this.nodeId = nodeId;
        this.parentNode = parentNode;
        this.hasToken = hasToken;
        this.requestQueue = new LinkedList<>();
    }

    public void requestToken(int requesterId, Map<Integer, Node> nodes) {
        if (hasToken) {
            System.out.println("Node " + nodeId + " sends token to Node "
+ requesterId);
            hasToken = false;
            nodes.get(requesterId).receiveToken();
        } else {
            System.out.println("Node " + nodeId + " forwards request to
Node " + parentNode);
            nodes.get(parentNode).requestToken(requesterId, nodes);
        }
    }

    public void receiveToken() {
        hasToken = true;
        System.out.println("Node " + nodeId + " received the token.");
        if (!requestQueue.isEmpty()) {
            int nextNode = requestQueue.poll();
            System.out.println("Node " + nodeId + " sends token to Node "
+ nextNode);
            hasToken = false;
        }
    }
}

```

```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();
        Map<Integer, Node> nodes = new HashMap<>();
        int rootNode = -1;
        for (int i = 0; i < n; i++) {
            System.out.print("Enter Node ID for node " + (i + 1) + ": ");
            int nodeId = sc.nextInt();
            System.out.print("Enter Parent Node ID for Node " + nodeId +
": ");

            int parentNode = sc.nextInt();
            System.out.print("Does this node have the token initially?
(true/false): ");
            boolean hasToken = sc.nextBoolean();
            nodes.put(nodeId, new Node(nodeId, parentNode, hasToken));
            if (hasToken)
                rootNode = nodeId;
        }

        System.out.print("Enter Node ID that requests the token: ");
        int requestingNode = sc.nextInt();
        nodes.get(requestingNode).requestToken(requestingNode, nodes);
    }
}

```

Output:

```
● PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXPS> javac Main.java
● PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXPS> java Main
Enter number of nodes: 5
Enter Node ID for node 1: 1
Enter Parent Node ID for Node 1: 3
Does this node have the token initially? (true/false): false
Enter Node ID for node 2: 2
Enter Parent Node ID for Node 2: 3
Does this node have the token initially? (true/false): false
Enter Node ID for node 3: 3
Enter Parent Node ID for Node 3: -1
Does this node have the token initially? (true/false): true
Enter Node ID for node 4: 4
Enter Parent Node ID for Node 4: 2
Does this node have the token initially? (true/false): false
Enter Node ID for node 5: 5
Enter Parent Node ID for Node 5: 2
Does this node have the token initially? (true/false): false
Enter Node ID that requests the token: 4
Node 4 forwards request to Node 2
Node 2 forwards request to Node 3
Node 3 sends token to Node 4
Node 4 received the token.
```